SafeMAP: Analysis-Guided Planning and Per-Function Validation for Safe C-to-Rust Translation

Jacob Sebastian Cyril[1*], Benita Jose Chalissery[1] and NewBegin M[1]

[1*]Department of Computer Science, CHRIST (Deemed to be) University, Bangalore, India.

*Corresponding author(s). E-mail(s): jacob.sebastian@mca.christuniversity.in;
Contributing authors: benita.jaison@christuniversity.in; new.begin@christuniversity.in;

Abstract

Automatic C-to-Rust translators either keep C's pointer conventions in unsafe Rust or rely on a large language model (LLM) whose output is hard to trust. SafeMAP translates a C function only when static analysis finds a safe, testable Rust interface for it, and refuses the rest with a recorded reason. Planned functions are synthesized by exact rules or a plan-guided LLM request with compiler-feedback repair. A translation is accepted only if it contains no `unsafe`, exposes no raw pointers, and matches the original C function on up to 1,000 generated inputs, with the C side run under AddressSanitizer and UndefinedBehaviorSanitizer. On ten held-out C libraries (356 translation units) selected under a sealed protocol, SafeMAP accepted 12–16 units across three runs under the sealed crate-level lint check. A declared amendment that applies the lint check per unit, re-scoring the same outputs, raised this to 36–37 units (10%). Under the amendment, the same LLM accepted 28–31 units when given SafeMAP's planned interfaces and 1–2 when given only the C sources; raw C2Rust accepted none. Most units were refused before translation because pointer ownership or aliasing could not be resolved statically (127) or because they use C types with no safe interface (116). The acceptance check rejected 93% of non-equivalent compiling mutants of accepted functions. Ablations attribute the accepted translations mainly to interface planning rather than to prompt guidance or repair. Per-function, oracle-checked acceptance is thus feasible on production C but covers about a tenth of it, and the recorded refusals show what currently blocks safe automatic migration.

Keywords: C-to-Rust migration, safe Rust, static analysis, large language models, differential testing, memory safety

1 Introduction

C remains common in systems software. Its memory model leaves programmers responsible for preventing buffer overflows, use-after-free errors, null dereferences, and data races. Safe Rust checks ownership and borrowing at compile time while retaining low-level control, so migrating C code to safe Rust is attractive. Translation is difficult because many C idioms have no direct safe-Rust equivalent: raw pointers, output parameters, manual allocation, nullable values, and integer error codes all require changes in representation and interface.

C2Rust preserves C behavior by generating Rust that uses raw pointers and unsafe operations (Immunant n.d.). Later work reduces this unsafety by static analysis, ownership inference, refactoring, and LLM rewriting (Emre et al. 2021; Hong 2023; Zhang et al. 2023; Ling et al. 2022; Nitin et al. 2026; Mohapatra et al. 2026). A second line asks LLMs to translate directly to safe Rust and checks the result with tests, fuzzing, or verification (Hong and Ryu 2025b; Zhou et al. 2025a; Eniser et al. 2025; Shetty et al. 2024; Zhou et al. 2025b; Yang et al. 2025; Sim et al. 2026; Shiraishi et al. 2026; Farrukh et al. 2026; Li et al. 2025). These systems generally aim to translate everything and then measure how much of the result is safe or correct.

SafeMAP takes the opposite position. It translates a function only when static analysis finds a safe Rust interface for it, and it accepts a translation only when that function, on its own, passes a behavioral check against the original C. Everything else is refused with a recorded reason. The question is therefore not how much unsafe code remains, but which translated functions can be trusted: how many functions of real C code can be migrated to safe Rust with per-function evidence of equivalence, and what prevents the rest. Our results locate SafeMAP's contribution in the planning step: an LLM told the planned safe interface produces checkable translations, whereas the same LLM given only the C code almost never does.

An earlier, frozen evaluation of SafeMAP on three C libraries accepted no units: defects in interface planning, rule matching, and harness construction each forced the outcome to zero (Section 4.1). This paper reports the corrected system, evaluated under a new sealed protocol on ten libraries that were not used during development. We ask:

- **RQ1:** How many translation units of production C libraries does SafeMAP accept as safe, behaviorally validated Rust?
- **RQ2:** Why are the remaining units not accepted?
- **RQ3:** How does SafeMAP compare with a one-shot LLM, with and without SafeMAP's planned interfaces, with its own rules alone, and with raw C2Rust, under one acceptance policy?
- **RQ4:** Which planning components contribute to acceptance?

- **RQ5:** How reliable is the acceptance decision, in terms of the harness's ability to reject wrong code and the variation across LLM runs?

This paper contributes:

- a translation pipeline that decides, per function and before any translation, whether a safe and testable Rust interface exists, and records a reason when it does not;
- a per-function acceptance procedure that compares each Rust function with the original C function under sanitizers, excluding inputs on which the C code has undefined behavior, and whose strength we measure by mutation analysis;
- a sealed, frozen evaluation on ten held-out libraries showing that planned interfaces raise a one-shot LLM from 1–2 to 28–31 accepted units and that SafeMAP's per-unit loop accepts 36–37 (under per-unit lint checking, a declared amendment), with repeated LLM runs, mutation analysis of the acceptance check, and component ablations; and
- a breakdown of 356 held-out units by the reason they could not be migrated, together with a public artifact for every decision.

2 Background and related work

2.1 C2Rust and safer-Rust refactoring

C2Rust translates C through a mostly syntactic process and is often used as a baseline or a migration starting point (Immunant n.d.). It preserves many C constructs with raw pointers and unsafe operations, so much of its output falls outside Rust's safe subset. Emre et al. found that raw pointers cause much of this unsafety (Emre et al. 2021) and later showed that Rust's borrow checker rejects many pointer uses that are in fact safe, which limits how far ownership inference can go (Emre et al. 2023). Studies of hand-written Rust find that even experienced developers use `unsafe` widely, mostly for interoperation and aliasing data structures (Astrauskas et al. 2020). Hong used static analysis to replace selected unsafe constructs in C2Rust output, including lock APIs and output parameters (Hong 2023). C2SaferRust combines C2Rust, static analysis, and LLM rewriting to reduce unsafe code in translated real-world programs (Nitin et al. 2026). Crown infers ownership models for C pointers and converts many of them to safe Rust types (Zhang et al. 2023). CRustS and Cp2SRust use different source-to-source transformations and transpiler designs (Ling et al. 2022; Mohapatra et al. 2026). Industry studies and surveys report that automatic translation still needs substantial manual work and testing (Hong and Ryu 2025a; Viirola et al. 2025).

2.2 LLM-based C-to-Rust translation

LLMs can produce idiomatic Rust, although correctness and type migration remain open problems. Hong and Ryu studied an LLM translator supplied with candidate signatures, caller and callee context, and compiler feedback (Hong and Ryu 2025b). C2RustTV checks LLM translations with generated tests and repair loops (Zhou et al. 2025a). Eniser et al. evaluate LLM translation through cross-language differential fuzzing and find lower success on more complex real-world samples (Eniser et al. 2025).

RustFlow studies prompting and iterative repair (Zhang et al. 2026). Valenzuela et al. examine compiler errors in direct C-to-safe-Rust translation (Valenzuela et al. 2025). RustMap combines program analysis and LLM support at project scale (Cai et al. 2025). Pointer knowledge graphs annotate code dependency graphs with ownership, mutability, and nullability facts to guide LLM translation of whole projects (Yuan et al. 2026), and His2Trans mines interfaces and idioms from earlier migrations of the same codebase (Wang et al. 2026c). Both supply the LLM with interface knowledge, which our results identify as the decisive input.

Several recent systems validate LLM translations against the C program. SmartC2Rust and SafeTrans repair translations iteratively from compiler errors and observed behavioral differences (Shiraishi et al. 2026; Farrukh et al. 2026). ACToR pairs a translator agent with an agent that builds differential fuzzers between the C and Rust binaries (Li et al. 2025), and TRAIL refines translation insights against executable counterexamples (Wang et al. 2026b). Syzygy translates units in dependency order and tests each one against its C counterpart with LLM-generated inputs (Shetty et al. 2024). SACTOR first produces interface-preserving Rust that may use `unsafe`, then refines it to idiomatic safe Rust, and checks both stages with end-to-end tests linked to the C code through the foreign-function interface (Zhou et al. 2025b). VERT checks a readable LLM translation against a WebAssembly-derived oracle with bounded model checking or property-based testing (Yang et al. 2025). ENCRUST wraps LLM translations in ABI-preserving shims and removes the remaining unsafe code with an agent (Sim et al. 2026), and Reboot translates whole interpreters to safe Rust through feature milestones with occasional user intervention (Wang et al. 2026a). CRUST-Bench supplies 100 C repositories with hand-written safe-Rust interfaces and tests (Khatry et al. 2025); those interfaces play the role that SafeMAP's planned signatures play here.

Table 1 places SafeMAP among these systems. SafeMAP differs in three ways. It decides per function, before translation, whether a safe interface exists, and records a reason when it does not instead of emitting `unsafe` code. Its oracle is the original C function itself, run under ASan and UBSan, and inputs on which C has undefined behavior are excluded rather than used as expected outputs. And acceptance is per function, so one failing unit does not hide the others. The cost of this conservatism is coverage, which Section 5 reports.

2.3 Undefined behavior and the C oracle

Differential testing compares two implementations on the same inputs and treats any disagreement as a defect (McKeeman 1998). It needs a trustworthy oracle. A C execution with undefined behavior does not define an expected output. ASan and UBSan detect many such executions (Serebryany et al. 2012; LLVM Project n.d.); SafeMAP runs every C oracle execution under both and discards inputs on which either reports an error. Miri can detect undefined behavior in Rust (Jung et al. 2026), but generated code that contains no `unsafe` cannot exhibit it, so SafeMAP does not run Miri.

Table 1 Validation and scope of recent C-to-Rust systems, as described by their authors.

System	Output	Behavioral oracle	Code it cannot translate safely
C2Rust (Immunant n.d.)	unsafe Rust	none	translated unsafely
Syzygy (Shetty et al. 2024)	safe Rust	C, LLM-generated inputs	repaired by LLM
SACTOR (Zhou et al. 2025b)	safe Rust	end-to-end tests via FFI	unsafe first stage
VERT (Yang et al. 2025)	safe Rust	Wasm-derived Rust oracle	regenerated
ENCRUST (Sim et al. 2026)	safe Rust	project test vectors	shims, then agent
SafeMAP	safe Rust	C under ASan/UBSan, per function	refused with reason

3 SafeMAP design

3.1 Acceptance policy

A translation unit is a C function, or a group of mutually recursive functions. SafeMAP accepts a unit if and only if all of the following hold:

1. it was admitted to the crate: it has the planned signature, contains no `unsafe`, raw pointers, `extern` blocks, or placeholders, and compiles together with the units already admitted;
2. the final crate compiles under `#![forbid(unsafe_code)]`, and its tests and Clippy pass (under amendment 3, Section 4.2, Clippy must report no error in the unit or in a unit it calls);
3. its public interface exposes no raw pointer types; and
4. its per-unit differential test passes (Section 3.4).

A skipped or inapplicable check never counts as a pass. A unit whose callee was not admitted is recorded as `dependency_missing`.

3.2 Pipeline

Figure 1 shows the pipeline. For each C project, SafeMAP:

1. recovers compile information and analyzes every function with libclang: signatures, pointer roles (read, write, index, length pairing, nullability), allocation, calls, and unsupported constructs;
2. groups mutually dependent functions into units and orders them by dependency;
3. screens each unit (`candidate_safe`, `manual_refactor_required`, `unsafe_required`, `unknown`); mutable global state makes a unit `manual_refactor_required`;
4. plans a safe Rust signature for each candidate. The planner reports every C type it cannot map to a safe, testable Rust type: opaque `void *` or `FILE *`, pointer-to-pointer, returned aliasing pointers, and structs whose pointer fields have unresolved

ownership. A unit with any such issue is refused at planning; no signature is fabricated. A struct pointer may be planned as a Rust struct of the struct's scalar fields when every use of the pointer in the unit is a scalar-field access, a null test, or an argument to a function that satisfies the same condition;

5. synthesizes each planned unit in dependency order, by an exact deterministic rule when one applies and otherwise by a plan-guided LLM request followed by up to three compiler-feedback repairs; and
6. validates the crate and each unit against the acceptance policy.

3.3 Deterministic rules and plan-guided synthesis

Deterministic rules cover scalar expressions, boolean and bit-reversal idioms, pointer-length and fixed-size arrays, scalar structs and field access, dependency-ordered internal calls, output parameters, return-code plus output-parameter to `Result`, nullable pointers, reductions, simple C string lengths, and simple allocation-to-`Vec` and `Box` patterns. A rule that renders a single expression applies only when the whole function body is that statement and the expression references only parameters and called functions.

When no rule applies, SafeMAP sends the LLM the C function, the macros, enums, and constant tables it references, the Rust definitions of the planned structs, the signatures of already-accepted callees, the analysis facts, and the migration plan, and asks for exactly the planned signature. The response is admitted only if it passes the policy check and compiles with the units already admitted; otherwise the compiler diagnostics are returned for up to three repairs.

Consider the LLVM development helper below. libclang finds two indexed pointers paired with `n` and no writes, so the plan maps each pointer-length pair to a shared slice, and the `slice_dot_product` rule applies:

```
double loop(float *x, float *y, long n) {
  double acc = 0.0;
  for (long i=0; i<n; ++i) acc += (double)x[i] * (double)y[i];
  return acc;
}

pub fn r#loop(x: &[f32], y: &[f32]) -> f64 {
  x.iter().zip(y.iter()).map(|(&a,&b)| (a as f64)*(b as f64))
   .fold(0.0_f64, |s,v| s + v)
}
```

The initial value `+0.0` is necessary: differential testing showed that `sum()` can disagree with C on the sign of zero.

3.4 Per-unit differential harness

For each admitted unit, SafeMAP generates inputs from a fixed seed, including integer literals mined from the C body and their neighbors, and embeds the same literal tables in a C harness and a Rust harness. Slice and array arguments are exact-size heap buffers, so an out-of-bounds read in C is caught by ASan rather than masked. The C harness includes the original translation unit and runs under ASan and UBSan with

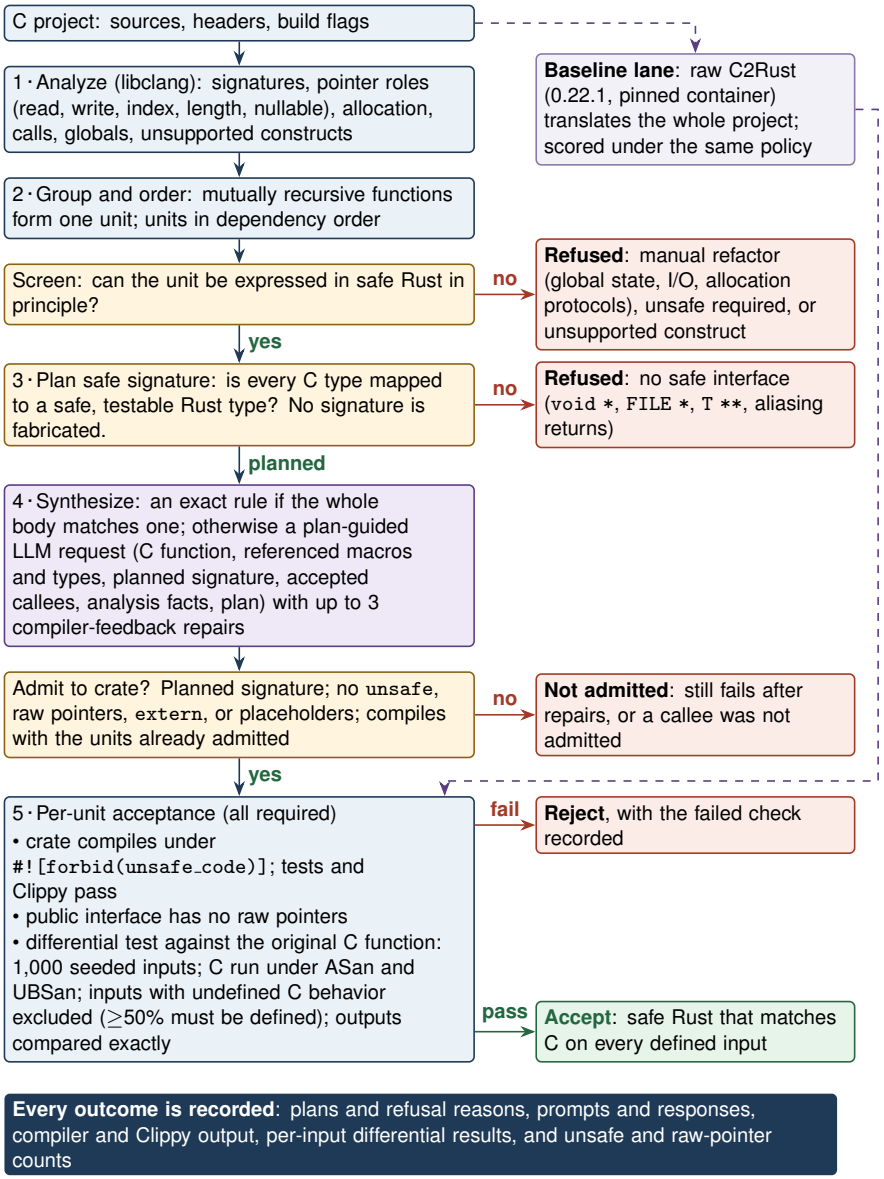


Fig. 1 SafeMAP pipeline. Units are refused at screening or at signature planning when no safe, testable interface exists. A planned unit is synthesized by an exact rule or, when none applies, by a plan-guided LLM request with compiler-feedback repair, and is accepted only if it passes every per-unit check, including a differential test against the original C function under sanitizers. Raw C2Rust is scored under the same policy. Every refusal and failed check is recorded with its reason

`-fno-sanitize-recover=all`; the Rust harness is built with overflow checks. Return values and every value written through a mutable parameter are compared exactly (floating point by bit pattern, strings byte by byte). An input on which the C oracle stops with a sanitizer report has no defined behavior and is excluded and counted. A defined C `abort` or `exit` must be matched by a Rust panic. A unit needs at least 50% defined inputs; otherwise it is recorded as `c_oracle_undefined_behavior`.

3.5 Baselines

Raw C2Rust translates every project in a pinned container (C2Rust 0.22.1, LLVM 14) and is measured under the same policy. *Rules only* runs the same pipeline without the LLM. In a single request, the *one-shot LLM* baseline receives the same work as SafeMAP: the C sources and headers of the project, the planned public signatures (with C names preserved), and the Rust struct definitions used by those signatures, with private helpers permitted. It does not receive any of SafeMAP's analysis facts, plans, rules, or repairs. Once top-level functions are made public, its output is scored using the same per-unit harness. Giving the interfaces isolates what SafeMAP adds beyond interface planning. The *plain LLM* baseline (amendment 2) measures the planning itself: the same request without the planned signatures and struct definitions, so the model sees only the C sources and the safety rules. Its output is scored the same way; a function with any other signature cannot be called by the harness and is recorded as `interface_mismatch`.

4 Evaluation methodology

4.1 Earlier evaluation

A first frozen evaluation (protocol v1) ran SafeMAP on inih, cJSON, and libcsv and accepted 0 of 142 units. Every one of six defects independently forced that result: the signature planner mapped C types incorrectly; rules fired on partial matches and emitted template code; struct definitions in headers were not collected; acceptance was all-or-nothing per project; the C2Rust setup shadowed libc headers; and the harness covered only scalar shapes. Because the v1 corpus was inspected while diagnosing these defects, it is used here only as development data, and the corrected system was evaluated on newly selected libraries under a new protocol.

4.2 Protocol

The v2 protocol fixed the selection rule, configurations, model, acceptance policy, and reporting before any SafeMAP execution on the held-out corpus. A library qualifies if it is a public Git repository with an OSI-approved license, its production code is one `.c` file plus headers that compiles against libc alone, the file has 150–3,500 lines, and it was not used during development. All functions in the file are retained. The protocol and corpus manifest were sealed and the implementation frozen at commit `62cf430` before the corpus was materialized; a materialization script refuses to run on any other commit. The full test suite passed at the freeze.

Table 2 Held-out corpus: ten libraries from two selection rounds.

Library	Round	C LOC	Functions	Units	Ptr. params	Complexity
microtar	Original	376	25	25	39	68
parson	Original	2494	144	138	206	633
sds	Original	1328	45	45	27	292
tinyexpr	Original	1025	48	37	28	306
argparse	Extension	395	13	13	27	101
json-parser	Extension	1186	10	10	15	227
printf	Extension	914	34	20	27	296
semver	Extension	638	33	33	29	127
tiny-AES-c	Extension	572	20	20	33	47
tiny-regex-c	Extension	528	18	15	23	126
Total		9456	390	356	454	2223

Four libraries were selected first: parson, sds, tinyexpr, and microtar. After their results were seen, we judged four libraries too few and added six more by the same rule (amendment 1): json-parser, tiny-regex-c, tiny-AES-c, semver.c, argparse, and printf. Nine candidates qualified in a fresh screen; six were chosen for domain diversity before any SafeMAP execution on them. The implementation, configuration, and modes were unchanged. We report the two rounds separately and pooled. After all results were seen we added one baseline (amendment 2), the plain LLM described above, run three times with the frozen implementation by a script that refuses to run if the implementation or configuration differs from the freeze.

Finally, after all results were seen, we changed how Clippy is applied (amendment 3). The sealed policy runs Clippy once per crate, so a single lint error rejects every unit in the crate (Section 5.1). Under amendment 3, each Clippy error is mapped to the function that contains it, and only that unit and the units that call it are rejected; an error outside every unit's function still rejects the crate. The same rule applies to SafeMAP and to the LLM baselines. No output was regenerated and no LLM was called: the archived crate of every held-out run was re-scored with the frozen checks and differential harness (`scripts/rescore_per_unit_clippy.py`). With the change switched off, the re-scoring reproduced the archived result of every run. Tables report the amended numbers, with the sealed number in parentheses wherever it differs. The development data and ablations were not re-scored. Table 2 characterizes the corpus.

4.3 Configuration

All LLM configurations, SafeMAP and baseline alike, use Google's `gemini-3.5-flash-lite` at temperature zero through its OpenAI-compatible API, with at most three repairs per unit, 1,000 generated inputs per unit, and seed 20260728. The evaluation used the provider's free tier. A request still refused after the client's retries stops the run rather than being scored as a failure; the run resumes after the quota resets, reusing every finished (project, configuration) result. Because LLM output can vary even at temperature zero (Ouyang et al. 2025), the LLM configurations were run three times on every held-out library with identical inputs; the

first run is the primary result. Tool versions: rustc and Clippy 1.99.1, Clang 23.1.3; C2Rust 0.22.1 with LLVM 14 in a pinned container.

Use of LLMs in this study. Apart from the evaluated pipeline's own calls to `gemini-3.5-flash-lite`, the authors used LLM assistants during the research. ChatGPT helped generate the 40 synthetic development microbenchmarks. ChatGPT and Claude assisted with implementing and testing SafeMAP, writing the evaluation scripts, and drafting and revising this manuscript. All AI-assisted code was reviewed by the authors and exercised by the project's test suite. Every reported number is produced by the committed scripts from the archived raw results, not written by hand. No LLM selected the held-out corpus or judged the correctness of a translation.

4.4 Development data

Development data comprises 40 authored microbenchmarks, six authored modules, ten LLVM test-suite programs pinned at commit `6cdc54e` (LLVM Project 2026), and the three v1 libraries. Their results informed the corrections, so they are not evidence of generalization. The component ablations are run on the v1 libraries with the frozen implementation and are reported as development data, which keeps the held-out LLM run within the provider's quota.

4.5 Mutation analysis

We performed mutation analysis (Jia and Harman 2011) to see if the per-unit check rejects incorrect code. Every accepted function of SafeMAP's held-out runs and of its v1-library run, including the functions accepted only under amendment 3, was subjected to single-operator mutations (relational, arithmetic, logical, boolean, and integer-literal changes). We retained the mutants that compile and reran each unit's differential test with 300 inputs from a different seed. Each surviving mutant was manually categorized as either an observation gap (the harness does not compare the differing output), an input gap (a distinguishing input exists but was not generated), or equivalent (no input can distinguish it).

5 Results

5.1 RQ1: Accepted units

Table 3 follows each held-out unit through the pipeline. In the first run, of 356 units, the screen marked 183 as candidates, SafeMAP generated Rust for 47, and accepted 37 (10.4% of all units; 20.2% of candidates). Under the sealed crate-level check it accepted 16 (4.5%). The original four libraries contributed 28 accepted units and the extension 9. Accepted units are small leaf functions: tinyexpr's arithmetic, comparison, and logical operators and its factorial and combinatorics helpers, a string hash, hexadecimal and UTF-8 helpers, rounding, error strings, version-string checks, regular-expression character classes, and the AES `xtime` step.

The whole difference between the sealed and amended results comes from tinyexpr. Its 24 generated units compiled, but two translated constants, `pi` and `e`, keep C's literal digits, which Clippy's default-deny `approx_constant` lint rejects. Under the

Table 3 SafeMAP (guided) outcome cascade per library (first run, per-unit Clippy; sealed result in parentheses where it differs). *Crate*: the unit passed `cargo check` and tests, and Clippy found no error in it or in its callees.

Library	Units	Candidate	Generated	Crate	Policy	Accepted
microtar	25	3	2	2	2	2
parson	138	72	4	4	4	3
sds	45	30	4	4	4	2
tinyexpr	37	32	24	22 (0)	22 (0)	21 (0)
argparse	13	0	0	0	0	0
json-parser	10	3	2	2	2	1
printf	20	18	0	0	0	0
semver	33	17	4	4	4	3
tiny-AES-c	20	1	1	1	1	1
tiny-regex-c	15	7	6	6	6	4
Total	356	183	47	45 (23)	45 (23)	37 (16)

sealed check, Clippy runs once per crate, so these two errors rejected all 24 units. Under amendment 3 they reject only `pi` and `e`. Of the other 22 units, 21 matched the C functions on every defined input, and one, `negate_logical_not`, disagreed with C.

5.2 RQ2: Why units are not accepted

Table 4 lists the recorded reasons. Most units never reach translation. The screen marked 159 functions as needing manual refactoring. For 127 of them, the analysis could not resolve pointer ownership or aliasing statically. The other 32 read or write mutable global state; 30 of these are in parson, whose allocator hooks `parson_malloc` and `parson_free` are global function pointers. 116 functions use a C type for which the planner finds no safe, testable interface. The most common case (48 functions) is a returned pointer that aliases caller-owned data, whose lifetime the planner does not model. In parson and sds the central types (`JSON_Value *`, `sds`) are opaque handles or point into the middle of an allocation, and `void *` and `char **` parameters are common. 36 functions require unsafe operations such as pointer arithmetic beyond a known length or casts between pointer types. Only a small fraction of losses occur after translation: 4 differential-test failures, 4 units whose C oracle had undefined behavior on most generated inputs, and 2 units with Clippy errors. Two of the undefined-behavior cases show a limit of signature-level planning: sds's `sdsll2str` writes up to 21 bytes into a caller buffer, a requirement its signature does not express, so every generated buffer overflowed. The other two overflow signed arithmetic on most inputs.

5.3 RQ3: Baselines

Table 5 compares the configurations. In the first run SafeMAP accepted 37 units, the one-shot LLM given the planned interfaces 28, the plain LLM 1, rules alone 6, and C2Rust none (sealed: 16, 9, 1, 1, and 0). SafeMAP accepted more units than the interface-given baseline in each of the three runs, 36–37 against 28–31 (Section 5.5). Under the sealed check the two ranges overlapped (12–16 against 9–15). In the first

Table 4 Recorded reasons for functions not accepted by SafeMAP (guided), all ten held-out libraries (first run, per-unit Clippy).

Reason	Functions
Manual refactor: pointer ownership or aliasing unresolved	127
No safe, testable interface for a C type	116
Requires unsafe (raw memory, casts)	36
Manual refactor: mutable global state	32
Unsupported construct (union, function pointer, asm)	12
Differential test failed	4
C undefined on most test inputs	4
Callee not translated	3
No synthesis plan	3
Clippy error in the unit or a callee	2

run the difference is concentrated in three libraries: tinyexpr (21 against 16), sds, and tiny-regex-c. Only tinyexpr's gap recurs in all three runs. On the other seven the two configurations are within one unit of each other, and on parson the baseline is one unit ahead. The plain LLM shows what the planned interfaces contribute: for four libraries its output used `unsafe` code or raw pointers and was rejected outright, and in the other six, of 43 planned functions, 24 were absent from the output and 12 had other signatures, so they could not be checked against the C function; 6 were in crates that failed compilation or Clippy, and 1, in semver, was accepted. SafeMAP used 40 LLM requests and the one-shot baseline 8 (one per library with planned interfaces). The one-shot baseline benefits from SafeMAP's interface planning: without the planned signatures, its output could not be tested function by function. Raw C2Rust output compiled for seven of ten libraries; for parson, sds, and printf it failed on the pinned toolchain because its `va_list` handling and packed-struct field references are rejected by current rustc. Where it compiled, it contained unsafe code and raw-pointer interfaces throughout, so no unit met the policy.

Development data, scored with the sealed crate-level check, show a similar pattern (Table 6): on the v1 libraries the two accepted 22 and 21 units. Only on the authored microbenchmarks, which the rules were written for, is SafeMAP clearly ahead (72 of 81 units against 35).

5.4 RQ4: Component ablations

Table 7 reports the ablations on the v1 libraries. The full system accepted 22 of 142 units. Removing safe-signature planning reduced this to 2, while raising LLM requests from 23 to 126, because the model must then invent interfaces that rarely compile with their callers. Removing idiom plans reduced acceptance to 3. Withholding analysis facts and plans from the prompt or removing compiler-feedback repair changed nothing (22), removing dependency grouping lost one unit (21), and removing pointer roles lost one (21). The planning that determines *what* interface each function gets, rather than the guidance given to the LLM, accounts for the accepted translations.

Table 5 Accepted units per configuration on the held-out libraries (first run, per-unit Clippy, amendment 3; sealed result in parentheses where it differs). One-shot: given the planned signatures; plain: C sources only (amendment 2). LLM calls: guided / one-shot. †C2Rust output did not compile.

Library	Units	Guided	One-shot	Plain	Rules	C2Rust	LLM calls
microtar	25	2	1	0	1	0	1 / 1
parson	138	3	4	0	0	0†	4 / 1
sds	45	2	0	0	0	0†	4 / 1
tinyexpr	37	21 (0)	16 (0)	0	5 (0)	0	17 / 1
argparse	13	0	0	0	0	0	0 / 0
json-parser	10	1	1	0	0	0	2 / 1
printf	20	0	0	0	0	0†	0 / 0
semver	33	3	3 (0)	1	0	0	5 / 1
tiny-AES-c	20	1	1	0	0	0	1 / 1
tiny-regex-c	15	4	2	0	0	0	6 / 1
Total	356	37 (16)	28 (9)	1	6 (1)	0	40 / 8

Table 6 Development data: accepted units per configuration.

Corpus	Units	Guided	One-shot LLM	Rules only	C2Rust
Microbenchmarks	81	72	35	35	0
Authored modules	25	24	18	18	0
LLVM programs	32	10	8	6	0
v1 libraries	142	22	21	0	0

Table 7 Component ablations on the v1 libraries (development data): accepted / all units.

Configuration	cJSON	inih	libcsv	LLM calls
Full system	15/109	1/10	6/23	23
Facts and plan withheld from prompt	15/109	1/10	6/23	22
Without pointer roles	15/109	0/10	6/23	23
Without safe signatures	2/109	0/10	0/23	126
Without dependency grouping	14/113	1/10	6/23	21
Without idiom plans	3/109	0/10	0/23	3
Without compiler-feedback repair	15/109	1/10	6/23	22

5.5 RQ5: Reliability of acceptance

Mutation analysis. We first mutated the functions accepted under the sealed check (first run of all ten held-out libraries, and the v1 libraries). Of 152 compiling mutants, the per-unit check rejected 135 (88.8%). Of the 17 survivors, 4 are equivalent: `>` replaced by `>=` directly after an equality test, the same change where both branches yield the same value on ties, a loop bound that a terminator written just before

Table 8 Accepted units across repeated LLM runs (identical inputs and configuration), with per-unit Clippy; sealed result in parentheses where it differs. One-shot: given the planned signatures; plain: C sources only.

Run	Libraries	Guided	One-shot	Plain
Run 1	10	37 (16)	28 (9)	1
Run 2	10	36 (15)	31 (15)	1
Run 3	10	36 (12)	30 (14)	2

already guarantees, and an unreachable `else`. We then mutated the 21 tinyexpr functions accepted only under amendment 3, in the same way. The check rejected 29 of their 34 compiling mutants. The 5 survivors, in `fac` and `ncr`, are equivalent: exhaustive checks over the reachable inputs show that each changes a comparison or starting value without changing any result. Over both sets, excluding the 9 equivalent mutants, the check rejected 164 of 177 non-equivalent mutants (92.7%); all 13 non-equivalent survivors are from the first set. Twelve survivors are input gaps: they change behavior only on a few specific values, such as one or two digits in sds's hexadecimal converter (six mutants), a string of exactly 255 or 256 bytes, or one error code, which uniformly generated inputs rarely hit. One is an observation gap: a copy routine's output differs only after the string terminator, which the harness, like a C caller, does not read. Targeted inputs from the C function's literals and comparison bounds would close most input gaps.

Repeated runs. Table 8 reports accepted units in three runs of the LLM configurations with identical inputs. SafeMAP accepted 37, 36, and 36 units, the one-shot baseline 28, 31, and 30, and the plain LLM 1, 1, and 2 (sealed: 16, 15, and 12 for SafeMAP; 9, 15, and 14 for the one-shot baseline). Nine of the ten libraries gave the same SafeMAP result in every run, against six for the one-shot baseline. Under the sealed check most of the variation came from semver: in one run of each configuration, one generated function failed a crate-level Clippy check and so rejected every planned unit of that crate, the same mechanism as tinyexpr. Per-unit Clippy removes this effect. The accepted sets are therefore stable at the function level, and the sealed results were sensitive to crate-level failures.

6 Discussion

Coverage is the limiting factor. The per-function acceptance check is feasible on production C and rejects most wrong code, but few functions reach it. Table 4 is the main finding. The obstacles are pointer ownership and aliasing that static analysis cannot resolve, returned pointers into caller-owned data, opaque pointer types, raw memory manipulation, and, in parson, global allocator hooks. Some of these could be planned: a pointer returned into an argument corresponds to a borrowed Rust return such as `fn f<'a>(s: &'a str) -> &'a str`. Others are API design questions that a function-local translator cannot answer alone. There, a migration would need to

change the library's interface before SafeMAP's checks apply, for example by turning an `sds` string into an owned type.

Interfaces matter more than prompts. On the held-out libraries the same model accepted 1–2 units from the C code alone and 28–31 when told the planned interfaces. SafeMAP's per-unit loop added a further 6–9 units in each run, most of them in one library. On the development libraries, removing signature planning from the guided loop reduces it from 22 to 2 units. Withholding analysis facts from the prompt changed nothing on the v1 libraries. This suggests that LLM translators benefit most from being told the target interface, which is consistent with CRUST-Bench's use of hand-written interfaces (Khatry et al. 2025).

Refusal as output. Every refused unit carries a reason and the plan that produced it. For a migration team this identifies which functions can be migrated mechanically and which need design work.

7 Threats to validity

Sample. Large applications or multi-file projects are not represented by ten small libraries with one production file each. The extension, which employed the same rule and frozen code, was chosen after the first four results were seen. We report the two rounds separately.

Model and variance. Three runs are insufficient to accurately quantify run-to-run variance, and just one inexpensive model was employed. The guided loop under per-unit Clippy accepted more than the one-shot baseline in each run, but its size is not established because the majority of the margin originates from a single library. Following the observation of the other outcomes, the plain LLM baseline and amendment 3 were added. Although it cannot alter the units rejected before translation, which account for the majority of losses, a stronger model might translate more of the planned units.

Oracle. Differential testing on generated inputs is evidence, not proof. Mutation analysis shows that 7.3% of non-equivalent compiling mutants survive, mainly where only a few specific inputs reach a branch. Excluding inputs with undefined C behavior removes inputs the sanitizers detect, not all undefined behavior.

Post-hoc amendment. The sealed policy ran Clippy on the whole crate, so one lint error rejected every unit in it (tinyexpr, and semver in one repeated run). Amendment 3 removes this, but it was decided after the results were seen. It changes only how the archived outputs are scored, applies the same rule to every configuration, and reproduces the sealed results when switched off. We report the sealed numbers beside the amended ones. A lint error outside every unit's function still rejects the whole crate.

No eligibility labels. We did not collect independent labels of which functions could be migrated safely, so we make no claim about the precision or recall of the screen; refusals are reported as recorded.

Toolchain. C2Rust 0.22.1's output does not compile with the pinned nightly on three libraries; another toolchain might compile it, but it would still use unsafe code and fail the policy.

Development data. The development corpora and the v1 libraries shaped the implementation; results on them are not evidence of generalization, and the ablations are run on them.

8 Conclusion

SafeMAP translates C functions to safe Rust only when it finds a safe interface and accepts them only when they match the original C under sanitizers, so every accepted function carries its own evidence of equivalence. On ten held-out libraries it accepted 12–16 of 356 units across three runs under the sealed crate-level lint check, and 36–37 when the same outputs were re-scored with the lint check applied per unit. Under that check, the same LLM accepted 1–2 units from the C code alone and 28–31 when given SafeMAP's planned interfaces; rules alone accepted 6 and raw C2Rust 0. These translations can be produced and verified thanks to the interface planning, and a minor further gain is added by SafeMAP's per-unit loop. The majority of units are rejected prior to translation because the function requires unsafe memory accesses, a C type lacks a safe interface, or pointer ownership or aliasing cannot be resolved statically. Oracle-checked, per-function migration is therefore possible, but it only covers around 10% of these libraries; its scope is determined by the extent to which a C library's interface already has a safe Rust equivalent. Future work should plan borrowed returns for functions that return pointers into their arguments, generate inputs from the C function's literals and bounds to close the remaining input gaps, and plan interface changes such as owned types for opaque handles. These changes should be evaluated on newly selected libraries, since the present corpus has now informed the analysis.

Statements and Declarations

Funding

No funding was received for conducting this study or preparing this manuscript.

Competing interests

The authors declare no competing financial or non-financial interests relevant to the content of this article.

Data availability

The source code, evaluation protocol, corpus manifests, configuration, frozen implementation record, raw results, and scripts that generate every table are archived on Zenodo (Cyril et al. 2026) and available in the SafeMAP repository (https://github.com/jscyril/SafeMAP); the implementation evaluated here is commit `62cf430`. The result rows of every run are in `reports/publication/v2`; the amendment-3 re-scoring, its reproduction check, and the added mutation analysis are in `reports/publication/v2/heldout/amendment3`, produced by `scripts/rescore_per_unit_clippy.py`. The script `scripts/generate_v2_manuscript_tables.py` regenerates every table from these

files. The held-out libraries are public repositories pinned to the revisions recorded in the manifests.

Ethics approval

Not applicable. The study involved no human participants.

Use of generative AI

Generative AI tools were used in this work. ChatGPT assisted with generating the synthetic development benchmarks. ChatGPT and Claude assisted with implementing and testing the software, writing the evaluation scripts, drafting and revising the manuscript, and refining its language. The evaluated pipeline itself calls Gemini as described in the paper. The authors reviewed and validated all AI-assisted code, data, and text, and remain responsible for the accuracy, originality, and integrity of the manuscript.

Author contributions

Jacob Sebastian Cyril: conceptualization, methodology, software, investigation, formal analysis, visualization, and writing—original draft. Benita Jose Chalissery: research guidance and manuscript review. NewBegin M: supervision and manuscript review. All authors approved the final manuscript.

References

Astrauskas V, Matheja C, Poli F, et al (2020) How do programmers use unsafe Rust? Proceedings of the ACM on Programming Languages 4(OOPSLA):136:1–136:27. https://doi.org/10.1145/3428204

Cai X, Liu J, Huang X, et al (2025) RustMap: Towards project-scale C-to-Rust migration via program analysis and LLM. In: Engineering of Complex Computer Systems: 29th International Conference, ICECCS 2025, pp 283–302, https://doi.org/10.1007/978-3-032-00828-2_16

Cyril JS, Chalissery BJ, M N (2026) Replication package for "SafeMAP: Analysis-guided planning and per-function validation for safe C-to-Rust translation". https://doi.org/10.5281/zenodo.23619725, dataset

Emre M, Schroeder R, Dewey K, et al (2021) Translating C to safer Rust. Proceedings of the ACM on Programming Languages 5(OOPSLA). https://doi.org/10.1145/3485498

Emre M, Boyland P, Parekh A, et al (2023) Aliasing limits on translating C to safe Rust. Proceedings of the ACM on Programming Languages 7(OOPSLA1). https://doi.org/10.1145/3586046

Eniser HF, Zhang H, David C, et al (2025) Towards translating real-world code with LLMs: A study of translating to Rust. arXiv:2405.11514

Farrukh M, Coskun B, Palit T, et al (2026) SafeTrans: LLM-assisted transpilation from C to Rust. In: Proceedings of the 1st International Workshop on Code Translation, Transformation, and Modernization (ReCode), https://doi.org/10.1145/3786180.3788317, arXiv:2505.10708

Hong J (2023) Improving automatic C-to-Rust translation with static analysis. In: 2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings, pp 273–277, https://doi.org/10.1109/ICSE-COMPANION58688.2023.00074

Hong J, Ryu S (2025a) Automatically translating C to Rust. Communications of the ACM 68(11):58–65. https://doi.org/10.1145/3737696

Hong J, Ryu S (2025b) Type-migrating C-to-Rust translation using a large language model. Empirical Software Engineering 30(1). https://doi.org/10.1007/s10664-024-10573-2

Immunant (n.d.) C2Rust: C to Rust translator. https://github.com/immunant/c2rust, accessed 2026-07-28

Jia Y, Harman M (2011) An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37(5):649–678. https://doi.org/10.1109/TSE.2010.62

Jung R, Kimock B, Poveda C, et al (2026) Miri: Practical undefined behavior detection for Rust. Proceedings of the ACM on Programming Languages 10(POPL). https://doi.org/10.1145/3776690

Khatry A, Zhang R, Pan J, et al (2025) CRUST-Bench: A comprehensive benchmark for C-to-safe-Rust transpilation. In: Conference on Language Modeling (COLM), arXiv:2504.15254

Li T, Li R, Wang B, et al (2025) Adversarial agent collaboration for correctness improvements of C to safe Rust translation. arXiv:2510.03879

Ling M, Yu Y, Wu H, et al (2022) In Rust we trust: A transpiler from unsafe C to safer Rust. In: 2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings, pp 354–355, https://doi.org/10.1109/ICSE-COMPANION55297.2022.9793767

LLVM Project (2026) LLVM test suite guide. https://llvm.org/docs/TestSuiteGuide.html, accessed 2026-07-23; corpus pinned to commit 6cdc54e005552e3444fa7402cd18a6e4b6db195d

LLVM Project (n.d.) UndefinedBehaviorSanitizer. https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html, accessed 2026-07-28

McKeeman WM (1998) Differential testing for software. Digital Technical Journal 10(1):100–107

Mohapatra V, Tripuramallu D, Behera AK, et al (2026) Cp2SRust: A transpiler for C/C++ to safer Rust. In: Innovations in Software Engineering Conference, pp 1–10, https://doi.org/10.1145/3796563.3796600

Nitin V, Krishna R, Valle LLd, et al (2026) C2SaferRust: Transforming C projects into safer Rust with neurosymbolic techniques. IEEE Transactions on Software Engineering 52(2):618–630. https://doi.org/10.1109/TSE.2025.3641486

Ouyang S, Zhang JM, Harman M, et al (2025) An empirical study of the non-determinism of ChatGPT in code generation. ACM Transactions on Software Engineering and Methodology 34(2). https://doi.org/10.1145/3697010

Serebryany K, Bruening D, Potapenko A, et al (2012) AddressSanitizer: A fast address sanity checker. In: 2012 USENIX Annual Technical Conference. USENIX Association, Boston, MA, pp 309–318, https://doi.org/10.5555/2342821.2342849, URL https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany

Shetty M, Jain N, Godbole A, et al (2024) Syzygy: Dual code-test C to (safe) Rust translation using LLMs and dynamic analysis. Preliminary version at LLM4Code 2025, arXiv:2412.14234

Shiraishi M, Cao Y, Shinagawa T (2026) SmartC2Rust: Iterative, feedback-driven C-to-Rust translation via large language models for safety and equivalence. In: Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE), https://doi.org/10.1145/3744916.3773259, arXiv:2409.10506

Sim H, Cho H, Shokri A, et al (2026) ENCRUST: Encapsulated substitution and agentic refinement on a live scaffold for safe C-to-Rust translation. arXiv:2604.04527

Valenzuela A, Gonzalez-Mallo M, Gutierrez C, et al (2025) From C to Rust: Evaluating LLM capabilities in transpilation through compilation errors. In: ISC High Performance 2025 International Workshops, pp 311–324, https://doi.org/10.1007/978-3-032-07612-0_24

Viirola E, Hummel J, Söderberg E, et al (2025) From C to Rust – how feasible is it? In: Product-Focused Software Process Improvement, pp 19–35, https://doi.org/10.1007/978-3-032-12092-2_2

Wang B, Paulsen B, Dodds J, et al (2026a) Mostly automatic translation of language interpreters from C to safe Rust. arXiv:2606.27122

Wang C, Gu X, Shi Y, et al (2026b) Translator vs. challenger: Adversarial agentic learning for C-to-Rust translation. arXiv:2609.15381

Wang S, Liu M, Ou G, et al (2026c) His2Trans: A knowledge-guided agentic framework for project-level C-to-Rust migration. arXiv:2603.02617

Yang AZH, Takashima Y, Paulsen B, et al (2025) VERT: Polyglot verified equivalent Rust transpilation with large language models. In: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), https://doi.org/10.1109/ASE63991.2025.00123

Yuan Z, Mao W, Chen Z, et al (2026) Project-level C-to-Rust translation via pointer knowledge graphs. Proceedings of the ACM on Software Engineering (FSE). https://doi.org/10.1145/3808169

Zhang H, David C, Yu Y, et al (2023) Ownership guided C to Rust translation. In: Computer Aided Verification, pp 459–482, https://doi.org/10.1007/978-3-031-37709-9_22

Zhang R, Zhang S, Xie L (2026) A systematic exploration of C-to-Rust code translation based on large language models: Prompt strategies and automated repair. Automated Software Engineering 33(1). https://doi.org/10.1007/s10515-025-00570-0

Zhou H, Luo Y, Zhang M, et al (2025a) C2RustTV: An LLM-based framework for C to Rust translation and validation. In: 2025 IEEE 49th Annual Computers, Software, and Applications Conference, pp 1254–1259, https://doi.org/10.1109/COMPSAC65507.2025.00158

Zhou T, Zhang Z, Lin H, et al (2025b) SACTOR: LLM-driven correct and idiomatic C to Rust translation with static analysis and FFI-based verification. arXiv:2503.12511